

1 – Race condition

ايتمى بتصير ؟

- بتصير وقت بكون في ضغط عفانكشن معينة يعني فرضا
عنا ٥ قطع بالمتجر اجا زبونين طلبو بنفس الوقت
اول زبون حينقص ١ و تاني زبون ١
فاللي حيصير اذا ما عالجنا ال race condition
انو حيصير الكوانتتي ٤ مو ٥ لانو التنين اشترفو بنفس الوقت و
حفظو بنفس الوقت

كيف منحلها ؟

- مستخدم شي اسمو Pessimistic Lock اللي هوة انو منخلي النظام
يفرض انو في اكثر من عملية شراء بنفس الوقت او أيا عملية تعديل
فشو منعهم وقتها ؟ اول مستخدم بفوت فبحط شي مثل قفل فبيمنع أيا
مستخدم تاني يعدل عيينما يحرر هادا القفل

كيف بدك متتبعو هالشى وين عم يصير بالكود

اول شي بتروحو عال routes

هون حيصير الاختبار

```
Route::prefix('inventory')->group(function () {  
    Route::post('/unsafe-decrease', [OrderController::class,  
    'unsafeDecrease']);  
    Route::post('/safe-decrease', [OrderController::class,  
    'safeDecrease']);  
});
```

بتروحو بعدها عالورد كونترولر ال unsafe هية بدون ما نعالج حالة
ال race ال safe مع معالجة

```

public function unsafeDecrease(Request $request): JsonResponse
{
    $validated = $request->validate([
        'product_id' => 'required|integer|exists:products,id',
        'amount'      => 'required|integer|min:1',
    ]);

    try {
        $result = $this->inventoryService->decreaseQuantityUnsafe(
            $validated['product_id'],
            $validated['amount']
        );
        return response()->json(['status' => 'ok', 'data' =>
$result]);
    } catch (Exception $e) {
        return response()->json(['status' => 'error', 'message' =>
$e->getMessage()], 400);
    }
}

```

ملاحظة : الكونترولر ماحطينا فيه أمور معقدة و معالجات لانو هاد

الصح ايا معالجة حتكون ب service

فهون منتقل لل service المسؤولة عن الانسيف

```

public function decreaseQuantityUnsafe(int $productId, int $amount):
array
{
    //RACE CONDITION: قراءة عادية بدون قفل
    $product = $this->inventoryRepository-
>findWithoutLock($productId);

    if (!$product) {
        throw new Exception("Product not found: #{$productId}");
    }

    if ($product->quantity < $amount) {
        throw new Exception("Insufficient stock (UNSAFE): only
{$product->quantity} left");
    }

    usleep(1000);
}

```

```

$product->quantity -= $amount;
$product->save();

return [
    'method'      => 'UNSAFE',
    'product_id'  => $productId,
    'new_quantity' => $product->quantity,
    'warning'     => 'Race Condition possible! Do NOT use in
production.',
];
}

```

مافي شي مو مفهوم اللا

```
usleep(1000);
```

مالو طعمة برمجيا اللا انو مشان نوضح المشكلة للمعيد علي عم يصير

مننقل لعلاج ال race condition

اتفقنا انو حنحط قفل

```

public function safeDecrease(Request $request): JsonResponse
{
    $validated = $request->validate([
        'product_id' => 'required|integer|exists:products,id',
        'amount'     => 'required|integer|min:1',
    ]);

    try {
        $result = $this->inventoryService->decreaseQuantitySafe(
            $validated['product_id'],
            $validated['amount']
        );
        return response()->json(['status' => 'ok', 'data' =>
$result]);
    } catch (Exception $e) {
        return response()->json(['status' => 'error', 'message' =>
$e->getMessage()], 400);
    }
}

```

مثل ما حكينا ما حنعالج شي هون ابدا

مننقل مباشرة لل service

```

public function decreaseQuantitySafe(int $productId, int $amount):
array
{
    return $this->inventoryRepository->transaction(function () use
($productId, $amount) {
        $product = $this->inventoryRepository-
>lockForUpdate($productId);

        if ($product->quantity < $amount) {
            throw new Exception("Insufficient stock: only
{$product->quantity} left");
        }

        $this->inventoryRepository->decrementStock($product,
$amount);

        Log::info("[SAFE] Product #{ $productId} decreased by
{$amount}. Remaining: " . ($product->quantity - $amount));
        return [
            'method'      => 'SAFE - Pessimistic Lock',
            'product_id'  => $productId,
            'new_quantity' => $product->fresh()->quantity,
        ];
    });
}

```

هون السر باستعمال ال transaction ليش؟

مشان تصير كل العمليات :

تمشي وحدة وحدة يا تنجح كلها

يا تفشل كلها حط شرح chatgpt لانو المفروض نكون فهمانيين
سطر سطر

.هذه الدالة هي النسخة "الآمنة" من خصم المخزون

هدفها:

1 باستخدام RACE CONDITION منع

- Transaction
- Pessimistic Lock

2 الدالة كاملة

```
public function decreaseQuantitySafe(int $productId, int $amount):  
array
```

3 معنى الترويسة

```
int $productId
```

رقم المنتج.

```
int $amount
```

الكمية المطلوبة للخصم.

```
: array
```

Array يعني الدالة سترجع.

4 بداية التنفيذ

```
return $this->inventoryRepository->transaction(function () use  
($productId, $amount) {
```

هذا أهم جزء بالبداية.

5 TRANSACTION ماذا يعني ؟

يعني:

ابدأ عملية محمية داخل قاعدة البيانات.

ينفذ Laravel تقريباً

```
BEGIN TRANSACTION;
```

6 USE ماذا يعني ؟

```
use ($productId, $amount)
```

Closure. لأننا داخل

Closure. حتى نمرر المتغيرات للـ

7 الآن ندخل إلى الجزء الأخطر

```
$product = $this->inventoryRepository->lockForUpdate($productId);
```

8 ماذا يفعل هذا السطر ؟

يقفل صف المنتج داخل قاعدة البيانات

غالبًا داخله

```
Product::lockForUpdate()->find($id);
```

9 SQL ويتحول إلى

```
SELECT * FROM products  
WHERE id = 1  
FOR UPDATE;
```

10 FOR UPDATE ماذا يعني ؟

يعني:

”أنا سأعدل هذا الصف، امنع أي شخص آخر من تعديله“

ماذا يحدث فعلياً؟ 11

المستخدم الأول 11.1

يدخل هنا:

```
lockForUpdate()
```

فيأخذ القفل.

المستخدم الثاني 11.2

يحاول الدخول لنفس المنتج

النتيجة 11.3

يتوقف هنا:

```
WAITING...
```

حتى ينتهي الأول.

لماذا هذا مهم؟ 12

لمنع:

13 RACE CONDITION

الآن التحقق من الكمية 14

```
if ($product->quantity < $amount)
```

إذا الكمية غير كافية.

15 ماذا يحدث هنا؟

```
throw new Exception(...)
```

يرمي خطأ

16 والأهم:

Transaction بما أننا داخل

ينفذ تلقائيًا Laravel

```
ROLLBACK;
```

17 ؟ ROLLBACK ماذا يعني

إلغاء كل شيء حدث داخل العملية

18 ثم يتم تحرير القفل تلقائيًا

وهذه نقطة مهمة جدًا

19 الآن الخصم الحقيقي

```
$this->inventoryRepository->decrementStock($product, $amount);
```

هنا يتم تعديل المخزون فعليًا

20 غالبًا داخله شيء مشابه

```
$product->quantity -= $amount;  
$product->save();
```

محمية Transaction لكن داخل

LOG ثم الـ 21

```
Log::info(...)
```

logs. يسجل العملية داخل

هذا الجزء مهم 22

```
($product->quantity - $amount)
```

يعرض الكمية بعد الخصم

النهاية 23

```
return [
```

إرجاع النتيجة.

هذا السطر مهم جدًا 24

```
$product->fresh()->quantity
```

25 FRESH() ما معنى ؟

إعادة جلب البيانات من قاعدة البيانات

لماذا نستخدمها؟ 26

\$product لأن قيمة

قد تكون قديمة داخل الذاكرة

فنريد أحدث قيمة بعد الحفظ

27 CLOSURE ماذا يحدث عند انتهاء الـ

ينفذ تلقائيًا Laravel

```
COMMIT;
```

28 COMMIT ماذا يعني ؟

- حفظ التعديلات نهائيًا
 - تحرير القفل
-

29 التسلسل الحقيقي الكامل

```
BEGIN TRANSACTION
↓
SELECT ... FOR UPDATE
↓
LOCK ROW
↓
CHECK STOCK
↓
UPDATE STOCK
↓
COMMIT
↓
UNLOCK ROW
```

30 مثال عملي كامل

31 الحالة الابتدائية

```
quantity = 5
```

32 المستخدم الأول

يريد خصم ٢

32.1 يدخل TRANSACTION

BEGIN

32.2 يأخذ Lock

FOR UPDATE

32.3 يخصص

5 → 3

33 بنفس اللحظة

المستخدم الثاني يريد خصم ١

33.1 ماذا يحدث؟

يتوقف عند

`lockForUpdate()`

ولا يستطيع القراءة

34 بعد انتهاء الأول

COMMIT

35 الآن الثاني يكمل

ويقرأ

3

وليس ٥

لذلك النظام أصبح آمناً 36

لماذا هذه الطريقة احترافية جداً؟ 37

لأنها تحقق:

كيف	الشيء
lockForUpdate	منع التضارب
Atomicity	transaction
COMMIT / ROLLBACK	سلامة البيانات
Pessimistic Lock	التزامن الآمن

أهم ٣ أسطر فعلياً 38

38.1 بدء العملية

`transaction()`

38.2 قفل الصف

`lockForUpdate()`

38.3 الحفظ النهائي

`COMMIT`

الخلاصة المبسطة جداً 39

الدالة تقول لقاعدة البيانات:

”اقفل المنتج مؤقتاً، لا تسمح لأحد بتعديله، سأتحقق من الكمية، أخصمها، ثم أحفظ التعديل وأفتح القفل“